

Laboratorio di Sicurezza dei Sistemi e Privacy

CdL Tecnologie dei Sistemi Informatici

Relazione di Progetto

Progetto di Secure Coding

Sviluppo e messa in sicurezza di un'applicazione prototipale
tramite AI e pratiche di Secure Coding

Autori:

Giovanni Morelli
Rei Mici

Anno Accademico 2025/2026

Indice

1	Il caso studio: Baudr e il Vibe Coding	3
1.1	Definizione di Vibe Coding	3
1.2	Analisi del Trattamento Dati e Criticità	3
1.3	L'incidente di sicurezza (Data Breach)	3
1.4	GDPR e Responsabilità	4
1.5	Situazione Attuale e Modello di Sviluppo	5
1.6	Considerazioni Critiche	5
2	Obiettivi del Progetto	6
3	Descrizione del Dominio Applicativo e Funzionalità	7
3.1	Dominio Applicativo	7
3.2	Funzionalità del Prototipo	7
4	Analisi della Baseline Insicura	8
4.1	SQL Injection	8
4.2	Gestione Insicura delle Sessioni e Broken Access Control	9
4.3	Crittografia Debole delle Password	10
4.4	Credenziali e Informazioni Sensibili Hardcoded	10
4.5	Caricamento di File Insicuro	11
4.6	IDOR (Insecure Direct Object Reference)	12
5	Implementazione del Secure Coding	14
5.1	SQL Injection	14
5.2	Gestione Insicura delle Sessioni e Broken Access Control	15
5.3	Crittografia delle Password con Bcrypt	17
5.4	Gestione Sicura dei Segreti tramite Variabili d'Ambiente	18
5.5	Caricamento di File Insicuro	19
5.6	IDOR (Insecure Direct Object Reference)	20
5.7	Difesa in Profondità (HTTP Headers)	22
6	Security Integration Tests	23
6.1	Architettura dei Test	23
6.2	Output di Debug e Analisi di Regressione	24
7	Appendice IA e Commenti	24
7.1	Elenco dei Prompt Utilizzati	24
7.2	Suddivisione del lavoro	25
7.3	Commenti finali	26

8	Tre ulteriori sfide per la sessione utente	27
8.1	Prima Sfida: Session Fingerprint Binding	27
8.1.1	Verifica della Vulnerabilità (Cookie Manipulation)	28
8.2	Seconda Sfida: Cifratura At-Rest nello Store delle Sessioni	29
8.2.1	Verifica della Cifratura (At-Rest Dump)	30
8.3	Terza Sfida: Controllo di Concorrenza	31
8.3.1	Verifica del Controllo di Concorrenza (Single Session Enforcement)	32

1 Il caso studio: Baudr e il Vibe Coding

Il 14 marzo 2026, lo streamer italiano Simone Buratti, conosciuto come "Grenbaud", con una community da milioni di utenti, lancia la propria Web App, chiamata **Baudr**.

Essa viene presentata come un servizio in grado di connettere via chat istantanea, in modo casuale, persone da ogni parte del mondo, con lo scopo di fare amicizia e conoscenza tra sconosciuti. Lo streamer afferma che il sito web lo ha creato personalmente lui, in una settimana di sviluppo, solamente con l'uso dell'AI e con 40 euro, probabilmente per l'abbonamento premium a qualche LLM, adottando il principio del "**Vibe Coding**".

1.1 Definizione di Vibe Coding

Il vibe coding è una pratica di sviluppo software, coniata dal ricercatore di AI Andrej Karpathy all'inizio del 2025, che rende la creazione di app più accessibile in particolare per coloro con esperienza di programmazione limitata. Questo approccio segna la fine del bisogno di anni di formazione tecnica, trasformando milioni di non programmatori in creatori capaci di lanciare applicazioni in pochi secondi.

Il flusso di lavoro si basa su un processo conversazionale in cui l'utente si concentra sull'obiettivo principale o sul quadro generale, mentre il ruolo principale passa dalla scrittura di codice riga per riga alla guida di un assistente AI che genera, perfeziona ed esegue il debug del codice vero e proprio.

Fonte: <https://cloud.google.com/discover/what-is-vibe-coding>
(Riadattato)

1.2 Analisi del Trattamento Dati e Criticità

La piattaforma prevede inoltre la raccolta di una quantità rilevante di dati personali, tra cui:

- Identificativi legati all'account Twitch;
- Informazioni anagrafiche, foto e preferenze personali;
- Contenuti generati dagli utenti (post, messaggi, interazioni);
- Dati derivati da analisi automatiche (compatibilità e punteggi di affinità).

Questa tipologia di trattamento dei dati colloca Baudr all'interno di un contesto ad alto rischio dal punto di vista della protezione dei dati personali.

1.3 L'incidente di sicurezza (Data Breach)

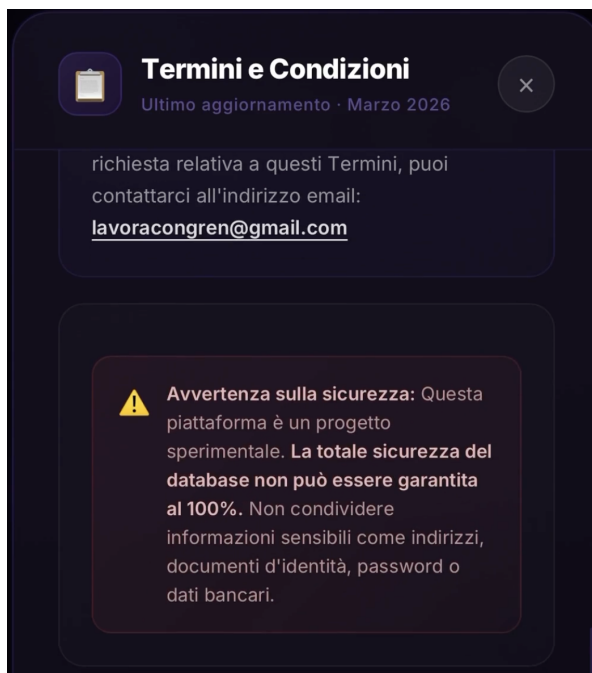
Dopo poche ore dal lancio, alcuni utenti hanno scoperto che il pannello di amministrazione della piattaforma Baudr era accessibile a chiunque scrivesse `/admin` nella barra del

browser. Questo accesso non autorizzato ha portato alla cancellazione di massa degli account, incluso quello dello stesso streamer, Grenbaud. L'episodio ha immediatamente sollevato il problema di un potenziale furto di identità e, cosa ancor più grave, di una potenziale violazione dei dati personali degli utenti iscritti al primo giorno di vita della piattaforma.

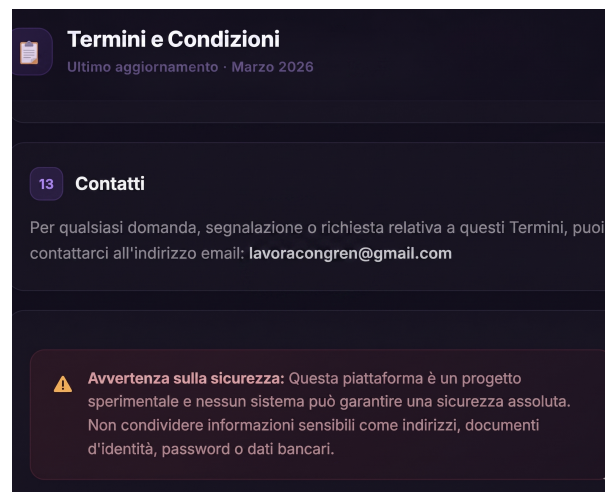
1.4 GDPR e Responsabilità

Il potenziale accesso non autorizzato al pannello di amministrazione della piattaforma Baudr configura un potenziale data breach. Ai sensi del GDPR, il titolare del trattamento, ovvero il creatore Simone Buratti, aveva l'obbligo di notificare tale violazione al Garante Privacy entro 72 ore dalla scoperta. La mancata protezione del pannello admin e l'assenza di controlli di sicurezza adeguati sono considerate indici di misure di sicurezza inadeguate, in violazione del principio di **accountability**.

A peggiorare la situazione, Grenbaud aveva inizialmente incluso nei Termini e Condizioni una clausola che ammetteva la non garanzia della sicurezza del database, sebbene successivamente modificata, rischiando di fornire una prova documentale della consapevolezza delle vulnerabilità da parte del titolare.



(a) Prima



(b) Dopo

Figura 1: Il prima e dopo dei Termini e Condizioni di Baudr.fun

Fonte: [Articolo su Substack](#)

1.5 Situazione Attuale e Modello di Sviluppo

Non risultano pubblicamente documentate sanzioni ufficiali già comminate nei confronti del suo creatore. Tuttavia, il sito tace in questo stato da dopo 3 giorni dal rilascio:

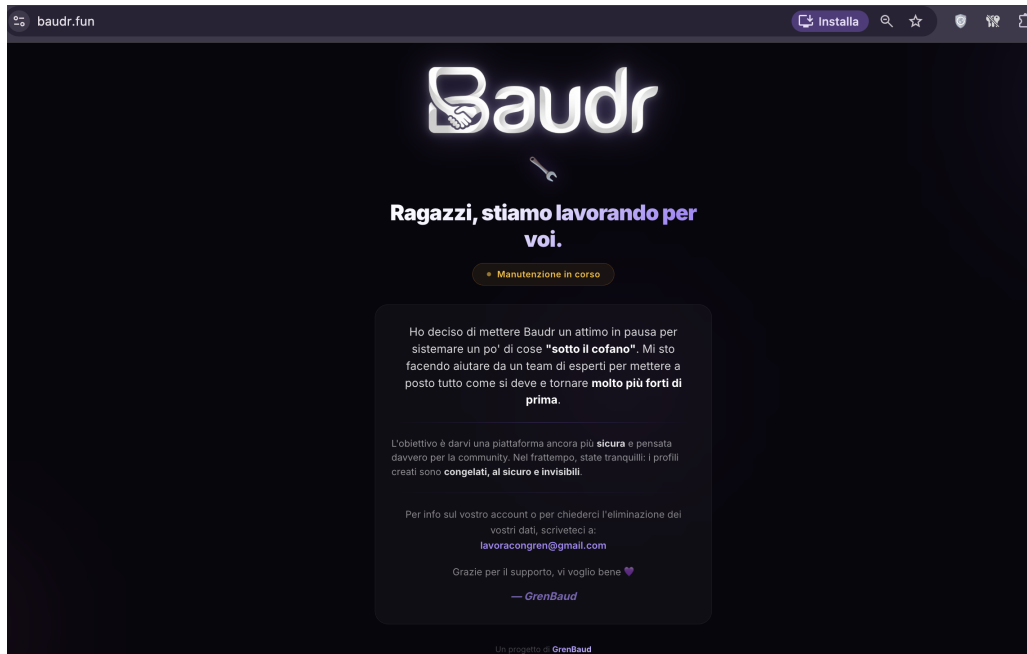


Figura 2: Screenshot di Baudr.fun

A quanto pare è stato affidato a un team di esperti, con il compito di implementare le parti di sicurezza e legislative per un futuro rilascio, ancora con data ignota.

1.6 Considerazioni Critiche

Il caso Baudr rappresenta un esempio dei limiti del vibe coding quando applicato a sistemi complessi e sensibili. L'utilizzo esclusivo dell'intelligenza artificiale per lo sviluppo, in assenza di revisione da parte di esperti, audit di sicurezza e consulenze legali, può portare alla realizzazione di applicazioni funzionali ma strutturalmente troppo fragili.

Fonti:

- <https://www.youtube.com/watch?v=nv6H7PDnc9c&t=6s>
- <https://www.youtube.com/watch?v=AzJ1L6d-di4>

2 Obiettivi del Progetto

Questo scenario si inserisce direttamente negli obiettivi del presente progetto, il quale si propone di replicare un'applicazione web adottando un approccio di *vibe coding*. In particolare, si intende simulare il comportamento di un utente medio, di età compresa tra i 20 e i 25 anni, con competenze informatiche di livello medio-basso.

Successivamente, l'applicazione realizzata verrà analizzata criticamente al fine di individuare eventuali vulnerabilità, sulle quali verranno poi applicate opportune tecniche di *secure coding*.

Durante la fase di sviluppo, la sicurezza dell'applicazione e le sue potenziali vulnerabilità non verranno mai esplicitamente menzionate all'interno dei prompt: sarà l'intelligenza artificiale a gestire autonomamente tali aspetti, privilegiando, a propria discrezione, la resa grafica rispetto alla sicurezza.

Per lo sviluppo dell'applicazione verrà utilizzato esclusivamente l'IDE *Antigravity*, sfruttando l'integrazione nativa con l'AI *Gemini 3.1 Pro (High)*. L'agente AI disporrà del pieno controllo su tutti i file del progetto, potendo eseguire operazioni CRUD in completa autonomia.

Al termine di ogni prompt, a seguito delle modifiche apportate al codice, è stato effettuato un commit sulla repository GitHub disponibile al seguente indirizzo:

<https://github.com/giovimori/MediVibe>

Ogni commit conterrà:

- nel titolo, il numero progressivo del prompt accompagnato da una breve descrizione;
- nella descrizione, il testo completo del prompt utilizzato.

All'interno della repository, per ogni intervento di *secure coding* effettuato, è stato inoltre creato un branch dedicato contenente la relativa modifica.

Il branch `initial-vulnerable-version` include il codice sorgente originale generato dall'AI, mentre il branch `main` contiene la versione finale dell'applicazione con tutte le correzioni di sicurezza implementate.

3 Descrizione del Dominio Applicativo e Funzionalità

3.1 Dominio Applicativo

Il dominio scelto è quello di una piattaforma di telemedicina, denominata **MediVibe**. L'applicazione consente l'interazione tra pazienti, medici e amministratori per la gestione di sintomi, visite e documentazione clinica.

La scelta è motivata dall'elevata criticità dei dati trattati, appartenenti alle categorie particolari previste dall'Art. 9 del GDPR. Questo rende il sistema ideale per analizzare vulnerabilità e impatti sulla sicurezza.

3.2 Funzionalità del Prototipo

Al fine di garantire i requisiti architetturali e funzionali forniti come traccia d'esame dal Prof., l'applicazione MediVibe include le seguenti funzionalità principali:

- **Sistema di autenticazione:** registrazione e login degli utenti con gestione delle credenziali;
- **Persistenza dei dati:** interazione con un database per la memorizzazione di profili, sintomi e documentazione clinica;
- **Gestione dei ruoli (RBAC):** distinzione tra pazienti, medici e amministratori, con differenti livelli di accesso;
- **Gestione dei contenuti utente:** possibilità per i pazienti di inserire sintomi e visualizzare le proprie informazioni sanitarie;
- **Controllo degli accessi:** ogni utente può accedere esclusivamente ai propri dati e documenti;
- **Gestione della sessione:** mantenimento dello stato di autenticazione tramite cookie o token.

In aggiunta, sono state implementate alcune funzionalità facoltative:

- **Upload di file:** caricamento di immagini o documenti PDF (es. referti medici);
- **Integrazione API:** simulazione di servizi esterni, come l'invio di email di conferma.

Dopo solamente 5 prompt, abbiamo ottenuto un sito perfettamente funzionante, secondo le specifiche di progetto. Passiamo all'analisi delle criticità analizzando a fondo il codice generato.

4 Analisi della Baseline Insicura

L'analisi del codice generato ha evidenziato la presenza di diverse vulnerabilità critiche, riconducibili alle principali categorie della OWASP Top 10.

ID	SECURITY RISK
A01:2025 =	Broken Access Control
A02:2025 ↑3	Security Misconfiguration
A03:2025 ↑3	Software Supply Chain Failures
A04:2025 ↓2	Cryptographic Failures
A05:2025 ↓2	Injection
A06:2025 ↓2	Insecure Design
A07:2025 =	Authentication Failures
A08:2025 =	Software or Data Integrity Failures
A09:2025 =	Security Logging and Alerting Failures
A10:2025 <i>New</i>	Mishandling of Exceptional Conditions

Figura 3: OWASP Top 10 2025

Di seguito vengono descritte le principali criticità individuate.

4.1 SQL Injection

Nel codice sorgente si utilizza la concatenazione diretta dei dati forniti dall'utente all'interno delle query SQL, combinando testo e istruzioni senza alcuna separazione tra contenuto e comando.

Il database non è in grado di distinguere tra dati e istruzioni: interpreta la query come un'unica struttura. Nel momento in cui l'utente inserisce caratteri speciali, come apici, commenti o operatori logici, questi possono alterare il significato originario della query, modificandone il comportamento.

Listing 1: Costruzione insicura della query

```
1 app.post('/login', (req, res) => {
2   const { email, password } = req.body;
3   const hashedPassword = crypto.createHash('md5').update(password)
4     .digest('hex');
5   const query = "SELECT * FROM users WHERE email =
```

```

6  _+email+_'+AND_password=_'" + hashedPassword + "";
7  });

```

Listing 2: Query SQL con input malevolo

```

1  SELECT * FROM users
2  WHERE email = 'admin@medivibe.com' --'
3  AND password = '[qualsiasiPassowrd]'

```

I due trattini indicano l'inizio di un commento. Di conseguenza, tutta la parte relativa al controllo della password viene ignorata, permettendo l'accesso a qualsiasi account senza conoscerne la password.

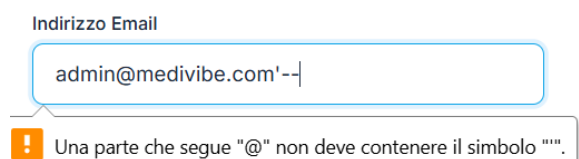


Figura 4: SQL Injection

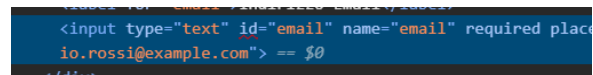


Figura 5: Input impostato su text per SQLi

Questa vulnerabilità rientra nella categoria **A05:2025 - Injection**.

4.2 Gestione Insicura delle Sessioni e Broken Access Control

L'applicazione si affida completamente ai dati memorizzati nei cookie lato client per la gestione dell'identità e dei privilegi utente. Questo approccio consente a un attaccante di modificare manualmente tali valori, ottenendo accessi non autorizzati a funzionalità riservate.

Listing 3: Gestione insicura dei cookie lato server

```

1  // Il server si fida ciecamente del client
2  res.cookie('user_id', row.id);
3  res.cookie('isAdmin', row.role === 'admin' ? 'true' : 'false');

```

Listing 4: Controllo di accesso basato su cookie non affidabili

```

1  app.get('/admin', (req, res) => {
2    if (req.cookies.isAdmin !== 'true') {
3      return res.send("Accesso Negato.");
4    }
5    // Accesso a dati sensibili
6  });

```

Questa vulnerabilità rientra nelle categorie **A01:2025 - Broken Access Control** e **A07:2025 - Identification and Authentication Failures**.

Name	Value	Domain	Path	Expires / ...
.AspNetCore.Antiforgery.ccl1zVfn15I	CfDJ8A9eBPoIHQNOKDoPo7QnYXaOJvLGjIMAdArl7WUnq_DOPh...	localhost	/	Session
.AspNetCore.Antiforgery.o7FwBgTQ-3A	CfDJ8A9eBPoIHQNOKDoPo7QnYXY2zroMExCSSIEsJQgnouqoiPR...	localhost	/	Session
.AspNetCore.Session	CfDJ8A9eBPoIHQNOKDoPo7QnYXY6sXBLIG3r7mAXO5PH4Sixlj...	localhost	/	Session
connect.sid	s%3A0r6fOD9W0COA8PtjTpX1CE-Y_RcW2nN1.1Q%2BBjh28giUA...	localhost	/	Session
isAdmin	true	localhost	/	Session
isDoctor	false	localhost	/	Session
user_id	3			

Figura 6: Screenshot dei cookies utente

4.3 Crittografia Debole delle Password

Analizzando il file `server.js` si è subito notato l'uso dell'algoritmo MD5 per l'hashing delle password degli utenti. MD5 è considerato crittograficamente obsoleto e non più idoneo alla protezione di credenziali. La sua elevata velocità di esecuzione permette ad un attaccante di tentare miliardi di combinazioni al secondo.

Inoltre, le password vengono hashate senza l'aggiunta di un salt (un valore casuale univoco). Questo rende il sistema vulnerabile ad attacchi tramite *Rainbow Tables*, ovvero database pre-calcolati di hash che permettono di risalire alla password in chiaro. Tale vulnerabilità è classificata come **A04:2025 - Cryptographic Failures**.

Listing 5: Utilizzo di MD5 senza salt nel codice originale

```
1 const hashedPassword = crypto.createHash('md5').update(password).
    digest('hex');
```

Dimostrazione: se due utenti si registrano con la stessa password, sul database apparirà lo stesso hash per entrambi (vulnerabilità alta a Rainbow Tables):

	name	email	password	symptoms	role
9	mario	mario@email.com	01b9622ff3ce2d88484089505fa99a63	NULL	patient
10	luigi	luigi@email.com	01b9622ff3ce2d88484089505fa99a63	NULL	patient

Figura 7: Screenshot della tabella 'users' del DB in cui sono salvati due utenti con la stessa password

4.4 Credenziali e Informazioni Sensibili Hardcoded

Revisionando il codice sorgente ho notato la presenza di *credenziali di accesso*, scritte in chiaro (hardcoded) direttamente all'interno del file `db.js` ('admin123' e 'doc123') utilizzate per il *seeding* iniziale del database. Inoltre il file `server.js` conteneva una chiave API per il servizio di invio email (SendGrid). Questa pratica di programmazione insicura (classificabile sotto **A04:2025 - Cryptographic Failures** e **A02:2025 - Security Misconfiguration**) espone criticamente il sistema: chiunque abbia accesso al

codice sorgente, ad esempio tramite una repository pubblica su GitHub, può sottrarre e abusare delle chiavi API o ottenere un accesso amministrativo non autorizzato sfruttando le credenziali di default note.

Listing 6: Chiave API hardcoded in server.js

```
1 const SENDGRID_API_KEY = "sg_test_12345_secret";
```

Listing 7: Password in chiaro nel seeding del database (db.js)

```
1 db.run("INSERT INTO users (name, email, password, role) VALUES (?, ?, ?, ?)",  
2       ['Admin', 'admin@medivibe.com', 'admin123', 'admin']);
```

4.5 Caricamento di File Insicuro

Il codice non controlla i dati inseriti dagli utenti, quindi accetta qualsiasi file senza verifiche. Questo permette a un attaccante di caricare file dannosi ed eseguire codice sul server, arrivando anche a rubare dati sensibili. Se viene inserito uno script malevolo nelle note cliniche, questo viene salvato e poi eseguito automaticamente nel browser di chi visita la pagina. In questo modo un attaccante può non solo rubare informazioni personali, ma anche modificare ciò che gli utenti vedono, nascondere dati o mostrare contenuti falsi.

Listing 8: Configurazione insicura dello storage e assenza di validazione in multer

```
1 const uploadDir = path.join(__dirname, 'public', 'uploads');  
2 const storage = multer.diskStorage({  
3   destination: function (req, file, cb) {  
4     cb(null, uploadDir)  
5   },  
6   filename: function (req, file, cb) {  
7     // Trust dell'input utente  
8     cb(null, file.originalname)  
9   }  
10 });  
11 const upload = multer({ storage: storage });
```

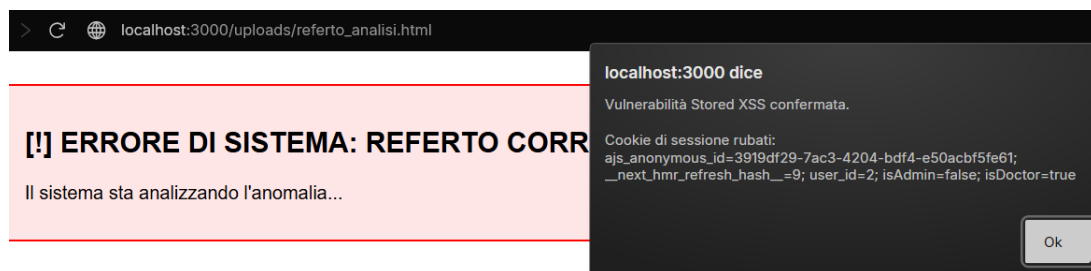


Figura 8: Caricamento di file malevolo e furto di dati sensibili.

Aggiungi o Aggiorna Anamnesi

```
<style>
body { background-color: black !important;
color: red !important; }
.card, .navbar { background-color: #111
!important; border-color: red !important; }
h1, h3, span, p { color: red !important; }
```

Salva nella Cartella

Figura 9: Script malevolo nel campo delle note cliniche.



Figura 10: Interfaccia modificata per tutti gli utenti.

Questa vulnerabilità rientra nelle categorie **A06:2025 - Insecure Design** e **A08:2025 - Software and Data Integrity Failures**.

4.6 IDOR (Insecure Direct Object Reference)

Un soggetto malevolo può accedere ai referti medici di altri utenti modificandone direttamente l'id all'interno della richiesta HTTP. Visto che il sistema non effettua un controllo adeguato sui permessi di accesso, non verifica se l'utente sia il reale proprietario del documento oppure un medico autorizzato. Di conseguenza, viene consentito l'accesso non autorizzato a informazioni sensibili appartenenti ad altri utenti.

Listing 9: Accesso ai dati senza verifica dell'appartenenza nell'endpoint dei referti.

```
1 app.get('/report', (req, res) => {
```

```

2  const reportId = req.query.id;
3  const isDoctor = req.cookies.isDoctor === 'true';
4
5  // Nessun controllo se r.user_id == req.cookies.user_id
6  const query = "SELECT r.*, u.name as patient_name FROM reports r
7  JOIN users u ON r.user_id = u.id WHERE r.id = " + reportId;
8
9  db.get(query, (err, report) => {
10   if (err || !report) return res.send("Referto non trovato.");
11   res.render('report', { report, isDoctor });
12 });

```

id	user_id	title
24	7	Dati Sensibili di Paz...

Figura 11: Dimostrazione che il referto con ID 24 appartiene all'utente con ID 7.

The screenshot shows a web application interface for 'MediVibe' with a patient record for 'Paziente Ingenuo'. The document is titled 'Documento Medico Ufficiale' with reference code '#MV-24-2026'. A section titled 'Dati Sensibili di Paziente da Violare' contains the text 'Nessuna nota clinica aggiuntiva presente sul referto.' and a button 'Scarica Copia Originale'.

On the right, the browser's developer console is open, showing the 'Application' tab. The 'Cookies' section for 'http://localhost:3000' is expanded, displaying the following cookies:

Name	Value	Domain	Path	Expires	Size	HttpOnly
_next_h...	9	lo...	/	S...	26	
ajs_anon...	3919df29-7ac3...	lo...	/	2...	52	
isAdmin	false	lo...	/	S...	12	
isDoctor	false	lo...	/	S...	13	
user_id	8	lo...	/	S...	8	

Figura 12: Modificando l'ID dei referti, è possibile accedere anche a quelli che non appartengono all'utente con ID 8.

Questa vulnerabilità rientra nella categoria **A01:2025 - Broken Access Control**.

5 Implementazione del Secure Coding

5.1 SQL Injection

Il codice originale univa istruzioni SQL e input utente in un'unica query tramite concatenazione diretta, permettendo a malintenzionati di alterarla: questo impedisce al DBMS di distinguere tra dati e comandi.

Listing 10: Costruzione insicura e dinamica dell'iniezione

```
1 // L'input "admin@medivibe.com" --" manipola la logica booleana
2 const query = "SELECT * FROM users WHERE email = " + email + " '
  AND password = '... '";
```

La mitigazione consiste nel far gestire l'interpretazione della query direttamente al database, invece che all'applicazione, dividendo il processo in due fasi.

Fase 1: Preparazione della Query e Separazione dei Domini

Conformemente alle buone pratiche di *Secure Coding*, applichiamo una netta separazione tra il dominio dei dati e il dominio del codice. Il database riceve solo la struttura pre-compilata della query con dei segnaposto (?), senza i valori reali. Il DBMS ne verifica la sintassi e prepara un piano di esecuzione fisso.

Listing 11: Definizione del Control Plane

```
1 const query = "SELECT * FROM users WHERE email = ?";
```

Fase 2: Inserimento dei dati

I dati vengono inviati separatamente dalla query. Visto la struttura è già definita, il database considera l'input solo come un valore, non come codice. Di conseguenza, eventuali caratteri speciali non hanno alcun effetto e vengono trattati semplicemente come testo.

Listing 12: Binding sicuro nel Data Plane

```
1 db.get(query, [req.body.email], async (err, row) => {
2   if (row && await bcrypt.compare(password, row.password)) {
3     /* Accesso sicuro */
4   });
```

5.2 Gestione Insicura delle Sessioni e Broken Access Control

1. Architettura dello Stato: Dal Client al Server

Nella versione originale, il mantenimento dello stato di autenticazione era delegato a cookie salvati in chiaro nel browser dell'utente (`user_id`, `isAdmin`). Per risolvere il problema è stato implementato un meccanismo di gestione delle sessioni lato server tramite il middleware `express-session`. I dati sensibili non lasciano mai il server; al browser del client viene rilasciato esclusivamente un **Session ID** (`connect.sid`) crittograficamente firmato (tramite una chiave segreta) per impedirne la manomissione.

2. Hardening del Session ID

Per garantire che il Session ID non diventasse un vettore di attacco, la configurazione del cookie è stata sottoposta a *hardening* applicando le best practice di sicurezza:

- Flag `HttpOnly`: Impostato a `true` per impedire ai client-side script (es. attacchi XSS) di accedere al token di sessione.

- Flag `SameSite`: Impostato a `lax` per mitigare tentativi di Cross-Site Request Forgery (CSRF).

3. Controllo degli Accessi, Least Privilege e Segregation of Duties

A livello di logica applicativa è stato implementato un modello RBAC (Role-Based Access Control) per garantire il **Principio del Privilegio Minimo (Least Privilege)** e la rigorosa **Separazione dei compiti (Segregation of Duties)**. Ogni singolo endpoint protetto (`/admin`, `/doctor`, `/dashboard`) è stato rifattorizzato per non accettare più parametri esterni, ma per verificare internamente la presenza di una sessione attiva e la corrispondenza del ruolo prima di procedere. Applicando il principio del *deny-by-default*, (Negazione di default), in caso di accesso non autorizzato la richiesta viene intercettata restituendo un errore HTTP 403 (Forbidden).

Infine, il logout distrugge fisicamente la sessione in memoria (`req.session.destroy()`), invalidando per sempre il token.

Confronto degli Snippet (Prima e Dopo)

Di seguito viene riportato un confronto tra l'implementazione originale (insicura) e quella corretta secondo le pratiche di *secure coding*, relativamente alla gestione della sessione in fase di autenticazione.

1. Inizializzazione della Sessione / Login

Codice Originale (Insicuro): I cookie venivano manipolati direttamente lato client.

Listing 13: Login insicuro - gestione cookie

```
1 res.cookie('user_id', row.id);
2 res.cookie('isAdmin', row.role === 'admin' ? 'true' : 'false')
  ;
```

Codice Sicuro (Mitigato): I dati sensibili vengono salvati lato server tramite sessione.

Listing 14: Login sicuro - utilizzo sessione

```
1 req.session.userId = row.id;
2 req.session.role = row.role;
```

2. Verifica dei Privilegi (Endpoint /admin)

Codice Originale (Insicuro): Controllo effettuato lato client, facilmente aggirabile.

Listing 15: Controllo accesso admin vulnerabile

```
1 app.get('/admin', (req, res) => {
2   if (req.cookies.isAdmin !== 'true') {
3     return res.send("Accesso Negato.");
4   }
5   // ...
6 });
```

Codice Sicuro (Mitigato): Controllo RBAC effettuato lato server.

Listing 16: Controllo accesso admin sicuro

```
1 app.get('/admin', (req, res) => {
2   if (!req.session.userId || req.session.role !== 'admin') {
3     return res.status(403).send("Accesso Negato.
4     Violazione dei privilegi rilevata.");
5   }
6   // ...
7 });
```

3. Distruzione della Sessione (Logout)

Codice Originale (Insicuro): Vengono eliminati solo i cookie lato client.

Listing 17: Logout insicuro

```
1 app.get('/logout', (req, res) => {
2   res.clearCookie('user_id');
3   res.clearCookie('isAdmin');
4   res.redirect('/login');
5 });
```

Codice Sicuro (Mitigato): La sessione viene distrutta lato server.

Listing 18: Logout sicuro

```

1 app.get('/logout', (req, res) => {
2     req.session.destroy((err) => {
3         res.clearCookie('connect.sid');
4         res.redirect('/login');
5     });
6 });

```

5.3 Crittografia delle Password con Bcrypt

Per risolvere i rischi legati alla crittografia debole, è stata implementata la libreria `bcrypt`. A differenza di MD5, `bcrypt` è un algoritmo progettato specificamente per le password (basato su *Blowfish*) che integra due meccanismi di difesa fondamentali:

- **Salt automatico:** Ogni password viene combinata con un valore casuale prima dell'hashing, garantendo che due utenti con la stessa password abbiano hash differenti nel database, rendendo inutili le Rainbow Tables.
- **Adaptive Cost Factor:** Permette di impostare un numero di round di cifratura (impostato a 10). Questo rende l'hashing lento lato server, ostacolando drasticamente gli attacchi di brute-force e dizionario.

Confronto degli Snippet (Prima e Dopo)

a) Fase di Registrazione (Hashing)

Codice Originale (Insicuro): Hashing immediato e veloce con MD5.

```

1 const hashedPassword = crypto.createHash('md5').update(
    password).digest('hex');

```

Codice Sicuro (Mitigato): Hashing asincrono con salt e fattore di costo.

```

1 const hashedPassword = await bcrypt.hash(password, 10);

```

b) Fase di Login (Verifica)

Codice Originale (Insicuro): Confronto diretto tra stringhe hashate (spesso tramite SQL injection vulnerabile).

```

1 const query = "SELECT * FROM users WHERE email = " + email +
    " AND password = " + hashedPassword + " ";

```

Codice Sicuro (Mitigato): Recupero dell'utente e verifica tramite funzione sicura dedicata.

```

1 db.get("SELECT_*_FROM_users_WHERE_email=_?", [email], async (
    err, row) => {
2     if (row && await bcrypt.compare(password, row.password)) {
3         // Password valida
4     }
5 });

```

5.4 Gestione Sicura dei Segreti tramite Variabili d'Ambiente

Per risolvere l'esposizione dei segreti nel codice sorgente, sono state sfruttate le **Variabili d'Ambiente** (*Environment Variables*), tramite libreria `dotenv`. Tutti i dati sensibili (chiavi API, password di default, chiavi di firma per le sessioni) sono stati estratti dal codice e confinati in un file di configurazione locale denominato `.env`.

Per prevenire futuri *data leak*, il file `.env` è stato escluso dal sistema di tracciamento inserendolo nel file `.gitignore`, garantendo che non venga mai caricato sulla repository pubblica. Inoltre, lo script di inizializzazione del database (`db.js`) è stato aggiornato per leggere le password di default dall'ambiente e applicare l'hashing tramite `bcrypt` prima dell'inserimento.

Confronto degli Snippet (Prima e Dopo)

a) Gestione Chiavi API

Codice Originale (Insicuro): Variabile valorizzata in chiaro nel codice.

```

1 const SENDGRID_API_KEY = "sg_test_12345_secret";

```

Codice Sicuro (Mitigato): Il valore viene letto dinamicamente dall'ambiente.

```

1 require('dotenv').config();
2 const SENDGRID_API_KEY = process.env.SENDGRID_API_KEY;

```

b) Inizializzazione Utenze di Default (Seeding)

Codice Originale (Insicuro): Password in chiaro visibili a chiunque legga il file di setup.

```

1 db.run("INSERT INTO users(name, email, password, role) VALUES
    (?, ?, ?, ?)",
2     ['Admin', 'admin@medivibe.com', 'admin123', 'admin']);

```

Codice Sicuro (Mitigato): Password lette dall'ambiente e sottoposte ad hashing crittografico prima del salvataggio.

```

1  const adminPass = process.env.DEFAULT_ADMIN_PASSWORD;
2  const hashedAdmin = await bcrypt.hash(adminPass, 10);
3
4  db.run("INSERT INTO users (name, email, password, role) VALUES
      (?, ?, ?, ?)",
5      ['Admin', 'admin@medivibe.com', hashedAdmin, 'admin']);

```

5.5 Caricamento di File Insicuro

Il modulo `multer` salva i file utilizzando direttamente il nome fornito dall'utente, senza effettuare alcuna verifica. Questo comportamento è critico perché si basa su un input completamente controllabile dall'esterno, in violazione del principio di Zero Trust. La mancata gestione del parametro `'file.originalname'` espone il sistema a vulnerabilità di Path Traversal: un attaccante può inserire nomi come `'../../server.js'` per tentare di sovrascrivere file di sistema o di configurazione. Inoltre, l'assenza di controlli sulle estensioni consente il caricamento di file potenzialmente eseguibili che, se raggiungibili dal server o dal browser, possono essere eseguiti o interpretati in modo malevolo.

Listing 19: Configurazione insicura dello storage e assenza di validazione

```

1  const uploadDir = path.join(__dirname, 'public', 'uploads');
2  const storage = multer.diskStorage({
3      destination: function (req, file, cb) {
4          cb(null, uploadDir)
5      },
6      filename: function (req, file, cb) {
7          // Trust dell'input utente
8          cb(null, file.originalname)
9      }
10 });
11 const upload = multer({ storage: storage });

```

Per risolvere il problema si interviene su due livelli: la separazione del nome del file dal sistema di archiviazione e l'applicazione di un filtro che consente solo file autorizzati.

Fase 1: Sostituzione del nome del file

Sostituiamo il nome originale del file con un identificativo casuale (come un hash o un UUID generato con `crypto.randomBytes(16)`). Questo riduce il rischio di manipolazione del nome o del percorso del file.

Listing 20: Generazione di identificativo univoco per il nome file

```

1  filename: function (req, file, cb) {

```

```

2   const uniqueSuffix = crypto.randomBytes(16).toString('hex');
3   const ext = path.extname(file.originalname).toLowerCase();
4   cb(null, uniqueSuffix + ext);
5 }

```

Fase 2: Input Validation e Whitelisting

Applicando i principi di Secure Coding per la validazione dell'input, abbiamo introdotto un controllo basato su **Whitelisting** (approccio *Default Deny*). Il filtro consente esclusivamente formati sicuri ed esplicitamente autorizzati (come PDF e immagini), bloccando di default tutti gli altri file potenzialmente malevoli.

Listing 21: Whitelist dei tipi di file consentiti

```

1  const fileFilter = (req, file, cb) => {
2    const allowedExtensions = ['.pdf', '.png', '.jpg', '.jpeg'];
3    const ext = path.extname(file.originalname).toLowerCase();
4
5    if (allowedExtensions.includes(ext)) {
6      cb(null, true);
7    } else {
8      cb(null, false);
9    }
10 };

```

5.6 IDOR (Insecure Direct Object Reference)

Un parametro controllato dall'utente (come 'reportId') viene utilizzato direttamente per accedere ai dati del database senza verificare se l'utente è autorizzato a visualizzarli. Il sistema considera erroneamente che l'autenticazione sia sufficiente per garantire l'accesso a qualsiasi informazione richiesta.

Listing 22: Implementazione vulnerabile con IDOR e SQL Injection.

```

1  app.get('/report', (req, res) => {
2    const reportId = req.query.id;
3    const isDoctor = req.cookies.isDoctor === 'true';
4
5    const query = "SELECT r.*, u.name as patient_name FROM reports_
6                  r_ " +
7                  "JOIN users u ON r.user_id = u.id WHERE r.id = "
8                  + reportId;
9
10   db.get(query, (err, report) => {

```

```

9         if (err || !report) return res.send("Referto non trovato.");
10        ;
11        res.render('report', { report, isDoctor });
12    });

```

Per risolvere il problema è necessario evitare l'uso diretto di input controllati dal client e introdurre controlli di autenticazione e autorizzazione più rigorosi.

Fase 1: Sessione sicura

Utilizziamo la sessione per recuperare l'identità e il ruolo dell'utente, evitando l'uso di dati modificabili come i cookie.

Listing 23: Recupero sicuro dei dati di sessione

```

1  const userId = req.session.userId;
2  const role = req.session.role;
3  const isAdmin = role === 'admin';
4
5  if (!userId) return res.redirect('/login');

```

Fase 2: Query parametrizzate

Introduciamo l'uso di query parametrizzate per evitare SQL Injection e separare i dati dalla struttura della query.

Listing 24: Query parametrizzata

```

1  const query = "SELECT r.*, u.name as patient_name, u.doctor_id " +
2                "FROM reports r JOIN users u ON r.user_id = u.id " +
3                "WHERE r.id = ?";
4
5  db.get(query, [req.query.id], (err, report) => {
6      if (err || !report) return res.send("Referto non trovato.");
7  });

```

Fase 3: Controllo dei permessi

Verifichiamo che l'utente sia autorizzato a visualizzare il contenuto prima di mostrarlo, controllando proprietà e ruolo.

Listing 25: Verifica dei permessi di accesso

```

1  const isOwner = report.user_id == userId;
2  const isDoctor = report.doctor_id == userId;

```

```

3
4 if (isAdmin || isOwner || isDoctor) {
5     res.render('report', { report, isDoctor: role === 'doctor' });
6 } else {
7     res.status(403).send("Accesso negato.");
8 }

```

5.7 Difesa in Profondità (HTTP Headers)

Per ridurre la superficie di attacco e implementare una strategia di *Difesa in Profondità*, è stata integrata la libreria `helmet` all'interno del server Express. Questa soluzione permette di configurare automaticamente header HTTP fondamentali che mitigano da subito molte possibili minacce basate sul browser e sulla rete.

Helmet sul branch sicuro configura le seguenti protezioni:

- **HTTP Strict Transport Security (HSTS):** Forza l'utilizzo di connessioni cifrate HTTPS, ostacolando attacchi di tipo Man-in-the-Middle ed impedendo il protocol downgrade.
- **X-Frame-Options:** Impostato su `SAMEORIGIN` per impedire il caricamento dell'applicazione all'interno di `<iframe>` non autorizzati, neutralizzando attacchi di Clickjacking.
- **Content Security Policy (CSP):** Stabilisce le sorgenti fidate da cui caricare script e risorse, riducendo drasticamente il rischio di attacchi Cross-Site Scripting (XSS).
- **X-Content-Type-Options:** Impostato su `nosniff` per bloccare tentativi di MIME sniffing da parte del browser, obbligandolo ad interpretare le risorse esclusivamente in base al Content-Type dichiarato dal server (essenziale per l'upload di referti).

Confronto degli Snippet (Prima e Dopo)

Codice Originale (Insicuro): Nessun header HTTP di sicurezza impostato per Express.

Codice Sicuro (Mitigato): Inizializzazione ed utilizzo globale del middleware.

Listing 26: Integrazione sicura di Helmet

```

1 const helmet = require('helmet');
2 app.use(helmet()); // Aggiunge difesa in profondit e header HTTP
   sicuri

```

6 Security Integration Tests

Per verificare l'efficacia del processo di hardening e prevenire future regressioni del livello di sicurezza, è stata implementata una serie di test automatici.

6.1 Architettura dei Test

1. **helper.js**: Gestisce il ciclo di vita del server di test avviando sequenzialmente un processo figlio Express su una porta libera, eseguendo la pulizia automatica del database SQLite ad ogni ciclo e gestendo la logica delle chiamate HTTP mock.
2. **sqli.test.js**: Verifica l'efficacia delle query parametrizzate contro tentativi di SQL Injection sul login ed endpoint dei referti.
3. **auth.test.js**: Controlla che richieste su rotte amministrative (/admin) o mediche (/doctor) con cookie locali contraffatti vengano intercettate e bloccate con successo.
4. **crypto.test.js**: Verifica direttamente l'integrità del database per accertare che gli hash delle password siano in formato Bcrypt e non MD5.
5. **secrets.test.js**: Scansiona il sorgente statico per scongiurare la presenza di API key o credenziali hardcoded.
6. **upload.test.js**: Esegue l'upload di un file .html fasullo testando che la blacklist respinga il caricamento.
7. **idor.test.js**: Esegue l'accesso come Paziente A tentando di visualizzare un documento del Paziente B, assicurandosi che il server neghi l'accesso con un codice di errore 403.
8. **concurrency.test.js**: Simula il controllo di concorrenza effettuando due login simultanei per lo stesso medico da postazioni diverse (Sessione A e Sessione B) e verifica che la Sessione A venga invalidata al tentativo successivo, mentre la Sessione B rimanga pienamente operativa.
9. **session_binding.test.js**: Simula un tentativo di session hijacking effettuando richieste con un cookie di sessione valido ma modificando l'IP o l>User-Agent del client, verificando che il server intercetti la discrepanza (HTTP 403) e invalidi la sessione.
10. **session_encryption.test.js**: Interroga l'endpoint di debug diagnostico per recuperare lo stato grezzo dei dati in memoria, verificando che i segreti siano cifrati in AES-256 at-rest nello store e che non vi siano informazioni in chiaro.

6.2 Output di Debug e Analisi di Regressione

Ogni test stampa a terminale i dettagli per il debug, indicando il payload inviato, il valore atteso e il valore effettivo rilevato.

- **Branch Vulnerabile:** Tutti e 10 i test falliscono fornendo dettagli completi sulle ragioni del fallimento (es. cookies di bypass creati, file arbitrari scritti su disco, report IDOR esposti, sessioni concorrenti attive, dirottamento della sessione consentito, sessioni memorizzate in chiaro).
- **Branch Hardened:** Tutti e 10 i test vengono superati con successo (`pass 10`), validando la robustezza del codice mitigato.

I test si avviano sequenzialmente tramite il comando:

Listing 27: Avvio dei test

```
1 npm test
```

7 Appendice IA e Commenti

7.1 Elenco dei Prompt Utilizzati

Di seguito sono riportati integralmente i prompt forniti all'Intelligenza Artificiale per la generazione dell'applicativo:

Prompt 1

Ciao, devo fare un progetto per l'università: un'app di telemedicina chiamata MediVibe. Voglio che la gente possa registrarsi e fare login. Segnalami errori di inserimento. Aggiungiamo poi i ruoli e un'area riservata: se sei paziente puoi scrivere i tuoi sintomi e quando vado nella mia bacheca fammeli leggere. Voglio mantenere l'accesso dopo il login. Infine, il professore vuole anche l'upload dei file e una chiamata API. Fammi un form per caricare le foto o i PDF dei referti e salvarli in una cartella. Poi metti una finta funzione che manda un'email.

Prompt 2

Ho visto che, caricando il progetto su GitHub, il processo richiede molto tempo: ci sono dei file che andrebbero ignorati? In caso affermativo, risolvi il problema.

Prompt 3

Ok, ottimo. Ora voglio migliorare la grafica del sito: come colore primario vorrei un azzurro chiaro. Voglio puntare al massimo sull'estetica, il sito deve apparire professionale, come un servizio certificato in ambito sanitario. Anche la UX è importante: desidero che i pazienti trovino facilmente le funzionalità all'interno del sito.

Prompt 4

Aggiungi un ruolo di dottore: ogni paziente deve essere associato a un dottore, mentre un dottore può seguire più pazienti. Nella pagina riservata ai dottori, ciascun dottore deve poter visualizzare la lista dei pazienti che segue, insieme ai loro documenti e alle informazioni caricate.

Aggiungi inoltre 3 dottori di default nel database per scopi di test e fai in modo che un paziente, al momento della registrazione o del primo accesso, possa scegliere il dottore che lo seguirà (una sola volta). Un dottore deve poter essere aggiunto manualmente tramite la dashboard dell'amministratore.

Prompt 5

Bug: quando accedo all'area del dottore e clicco sul bottone "vedi scheda" relativo a un paziente, il dottore viene reindirizzato all'area di un paziente normale, perdendo così i propri privilegi. Risolvi questo problema.

7.2 Suddivisione del lavoro

Sviluppo del prototipo applicativo e prompt engineering:

- Attività svolta insieme da entrambi i membri del gruppo.

Analisi delle criticità e implementazione delle soluzioni:

- **Giovanni Morelli:**

- 4.2 Gestione Insicura delle Sessioni e Broken Access Control
- 4.3 Crittografia Debole delle Password
- 4.4 Credenziali e Informazioni Sensibili Hardcoded

- **Rei Mici:**

- 4.1 SQL Injection

- 4.5 Caricamento di File Insicuro
- 4.6 IDOR (Insecure Direct Object Reference)

Stesura della relazione:

- Attività svolta insieme da entrambi i membri del gruppo.

7.3 Commenti finali

L'esperimento condotto in questo progetto ha permesso di evidenziare chiaramente i limiti dell'approccio del *vibe coding* e l'inadeguatezza degli attuali LLM nella stesura autonoma di codice sicuro.

Analizzando il codice generato nei primi 5 prompt si può notare come l'agente IA sia stato vittima di quelle che possiamo definire "**allucinazioni di sicurezza**". L'IA non ha inventato funzionalità inesistenti, ma ha ipotizzato un ambiente in cui tutti gli attori sono benevoli e le interazioni sono prive di minacce.

Non si esclude che, adottando un approccio diverso e impersonificando uno sviluppatore esperto nei prompt, sarebbe stato possibile mitigare gran parte di queste vulnerabilità già in fase di generazione del codice. Tuttavia, nel contesto di questo progetto, ci si è volutamente immedesimati in un utente con competenze di programmazione medio-basse: in tale scenario, l'applicazione ottenuta dopo soli cinque prompt risulta sì funzionante, ma presenta criticità significative dal punto di vista della sicurezza. Nello specifico, sono emerse tre tendenze critiche:

- 1. Orientamento alla sola funzionalità:** L'IA è programmata per assecondare l'utente e raggiungere l'obiettivo nel minor tempo possibile (es. "Voglio che la gente possa fare login"). Per farlo, tende a ignorare sistematicamente i requisiti non funzionali legati alla sicurezza, a meno che questi non vengano esplicitamente richiesti nel prompt. L'IA ha scelto la via più semplice: concatenare stringhe SQL piuttosto che usare i *prepared statements*, e usare i cookie in chiaro piuttosto che configurare un middleware di sessione.
- 2. Fiducia Implicita (Assenza di Zero-Trust):** L'IA ha delegato il controllo degli accessi al client (tramite il cookie `isAdmin`), dimostrando di non possedere una reale "comprensione architetturale" delle reti. Ha generato codice che presupponeva l'impossibilità, da parte dell'utente, di manipolare il proprio browser, violando il principio cardine della sicurezza web: *Never trust user input*.
- 3. Ricorso a Pattern Obsoleti (Il problema del Training Data):** L'utilizzo dell'algoritmo MD5 senza *salt* per l'hashing delle password è forse l'allucinazione più grave. Sebbene MD5 sia deprecato da oltre un decennio, la sua enorme diffusione in vecchi tutorial, forum e repository presenti nei dati di addestramento dell'LLM porta l'IA a considerarlo ancora uno standard accettabile per la risoluzione rapida del problema.

Conclusioni

Il progetto dimostra che l'Intelligenza Artificiale Generativa è uno strumento estremamente potente per la prototipazione rapida e la definizione della UI/UX. Tuttavia, se lasciata senza supervisione (*vibe coding* puro), tende a produrre codice con gravi problemi di sicurezza e strutturali.

A nostro parere, la stesura di software in ambiti sensibili, come quello sanitario (Art. 9 GDPR), richiede obbligatoriamente l'intervento umano per l'attività di messa in sicurezza. L'IA può agire in modo eccellente per la scrittura del codice base, ma il ruolo di "Security Architect" e la responsabilità dell'applicazione delle **linee guida di sviluppo sicuro** (es. **SEI CERT SCS**) e dei principi OWASP restano, e resteranno, una prerogativa del fattore umano.

8 Tre ulteriori sfide per la sessione utente

Nell'ambito di questa Proof of Concept (PoC), abbiamo aggiunto le tre sfide consigliate dal Prof. relative alla sicurezza e alla gestione del ciclo di vita della sessione.

8.1 Prima Sfida: Session Fingerprint Binding

La prima sfida consiste nel mitigare il rischio di *Session Hijacking* (furto di sessione), associando la sessione attiva (`connect.sid`) ad un fingerprint applicativo basato su indirizzo IP e User-Agent del client. In questo modo, se un attaccante dovesse rubare il cookie di sessione della vittima, non potrebbe riutilizzarlo facilmente da un dispositivo o contesto client differente.

Per risolvere questa vulnerabilità, abbiamo implementato i seguenti componenti architetturali:

- **Fingerprint al Login:** Durante la fase di autenticazione andata a buon fine in `server.js`, l'indirizzo IP del client (estratto tramite l'header `x-forwarded-for` o la proprietà `req.socket.remoteAddress`) e l'User-Agent dell'header HTTP sono registrati nella sessione dell'utente (`req.session.clientIp` e `req.session.clientUserAgent`).
- **Middleware di Controllo:** È stato introdotto il middleware globale `sessionFingerprintCheck`. Ad ogni richiesta HTTP effettuata da un utente autenticato, il middleware confronta l'IP e l'User-Agent correnti del client con quelli associati alla sessione al momento del login.
- **Invalidazione Immediata:** Qualora venga rilevata una discrepanza (ad esempio a causa di una modifica dell'User-Agent o dell'indirizzo IP del mittente), il

middleware intercetta la violazione di sicurezza, rimuove l'associazione di sessione dal database (`active_session_id = NULL`), distrugge la sessione lato server (`req.session.destroy`), cancella il cookie sul client (`connect.sid`) e nega l'accesso con codice di stato 403 Forbidden.

Questa soluzione aumenta significativamente la sicurezza contro il riutilizzo illecito del cookie di sessione da parte di terzi, pur potendo generare falsi positivi in presenza di cambiamenti legittimi dell'indirizzo IP (ad esempio durante il passaggio da una rete Wi-Fi ad una rete mobile).

8.1.1 Verifica della Vulnerabilità (Cookie Manipulation)

Per dimostrare l'efficacia del meccanismo di protezione ed evidenziare la vulnerabilità presente nella versione iniziale (`initial-vulnerable-version`), è stato effettuato un test manuale basato sulla manipolazione lato client dei cookie.

Nel sistema vulnerabile, l'autenticazione è legata a cookie non firmati e facilmente falsificabili inviati direttamente dal browser (come `user_id` ed `isAdmin`). Per dimostrare la vulnerabilità, è stata avviata l'applicazione sul branch `initial-vulnerable-version` e, senza effettuare alcuna autenticazione, sono stati utilizzati gli strumenti per sviluppatori del browser (F12) per inserire manualmente i cookie `user_id` con valore 1 e `isAdmin` impostato su `true`. Successivamente è stata effettuata una richiesta diretta alla pagina riservata `/admin`, verificando l'aggiramento dei controlli di accesso e l'ottenimento dei privilegi amministrativi.

Come documentato in Figura 13, l'assenza di controlli di sessione lato server e la cieca fiducia nei cookie client-side consentono all'attaccante di bypassare completamente il meccanismo di autenticazione, ottenendo accesso immediato al pannello amministrativo e compromettendo le informazioni riservate degli utenti.

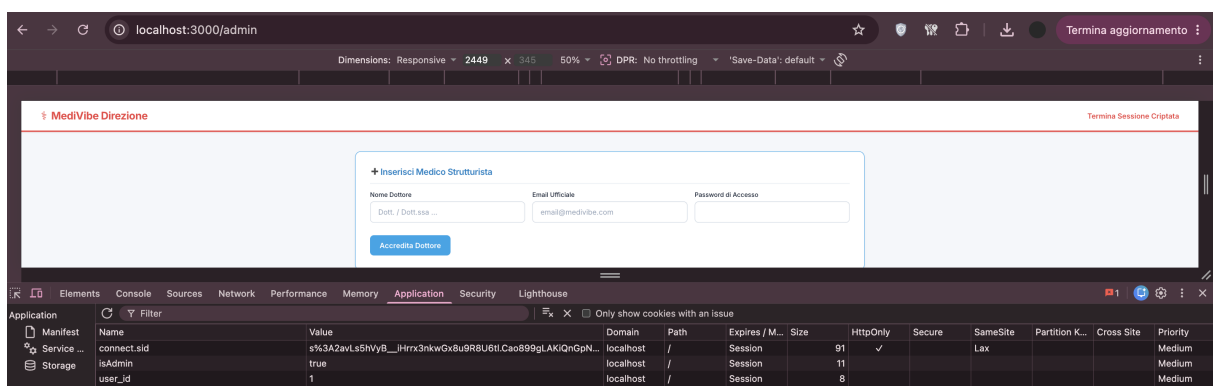


Figura 13: Accesso non autorizzato all'area amministratore tramite iniezione manuale dei cookie `user_id` e `isAdmin` sul client nella versione vulnerabile.

8.2 Seconda Sfida: Cifratura At-Rest nello Store delle Sessioni

La seconda sfida riguarda la sicurezza dei dati di sessione (come l'ID utente, il ruolo e i fingerprint IP/User-Agent) memorizzati a riposo (*at-rest*) nello store delle sessioni. Se un attaccante riuscisse ad accedere o ad effettuare un dump della memoria dello store, potrebbe leggere in chiaro le sessioni attive ed ottenere informazioni di autenticazione, ruoli applicativi e altri attributi associati agli utenti autenticati.

Per mitigare questa minaccia, abbiamo implementato un meccanismo di cifratura simmetrica trasparente **AES-256-CBC** per lo store di **express-session**.

Sebbene la PoC utilizzi **MemoryStore**, la soluzione è stata progettata in modo trasparente rispetto allo store sottostante e risulta applicabile anche a store persistenti come Redis.

- **Cifratura in Scrittura:** Abbiamo realizzato un wrapper applicativo che effettua l'override del metodo `store.set`. Prima che i dati della sessione vengano scritti in memoria o inviati ad uno store esterno, il payload viene serializzato in formato JSON e cifrato tramite algoritmo `aes-256-cbc` con una chiave derivata in modo sicuro (`crypto.scryptSync`) a partire dal parametro d'ambiente `SESSION_ENCRYPTION_KEY`. Lo store riceve quindi esclusivamente un payload cifrato del tipo `{ encryptedPayload: iv + ":" + ciphertext }`.
- **Decifratura in Lettura:** È stato effettuato l'override del metodo `store.get`. Quando Express richiede i dati di sessione per validare un client, il wrapper intercetta il record cifrato dallo store, lo decifra, lo deserializza e lo restituisce in chiaro all'applicazione, rendendo l'intera operazione trasparente alla logica di business.

Per semplicità implementativa è stato utilizzato l'algoritmo **AES-256-CBC**. In un ambiente produttivo sarebbe preferibile adottare una modalità di cifratura autenticata, come **AES-256-GCM**, in grado di garantire sia la confidenzialità sia l'integrità dei dati memorizzati.

Snippet del Codice di Hardening (Wrapper Crittografico):

Listing 28: Override dei metodi set e get dello store per la cifratura

```
1 const sessionStore = new session.MemoryStore();
2 const originalGet = sessionStore.get.bind(sessionStore);
3 const originalSet = sessionStore.set.bind(sessionStore);
4
5 // Scrittura: serializza in JSON e cifra i dati prima del
  salvataggio
6 sessionStore.set = (sid, sessionData, callback) => {
7   const encrypted = encryptSession(JSON.stringify(sessionData));
8   originalSet(sid, { encryptedPayload: encrypted }, callback);
9 };
```

```

10
11 // Lettura: recupera il payload, lo decifra e lo restituisce in
    chiaro all'app
12 sessionStore.get = (sid, callback) => {
13     originalGet(sid, (err, sessionData) => {
14         if (sessionData && sessionData.encryptedPayload) {
15             const decrypted = decryptSession(sessionData.
                encryptedPayload);
16             return callback(null, JSON.parse(decrypted));
17         }
18         callback(err, sessionData);
19     });
20 };

```

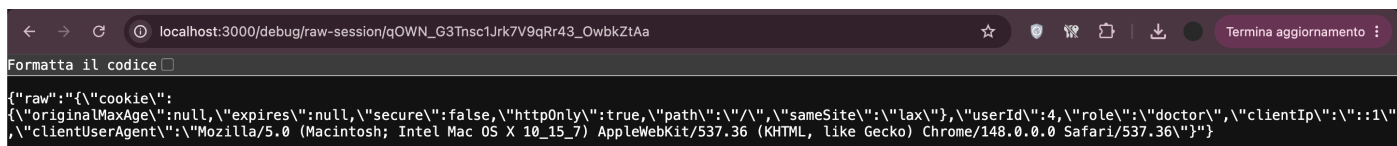
8.2.1 Verifica della Cifratura (At-Rest Dump)

Per verificare l'effettiva cifratura at-rest dei dati di sessione ed evidenziare il contrasto con una memorizzazione insicura in chiaro, viene sfruttato l'endpoint di debug `/debug/raw-session/:sid` (attivo solo in modalità di test: `NODE_ENV=test`).

Il test manuale si articola come segue:

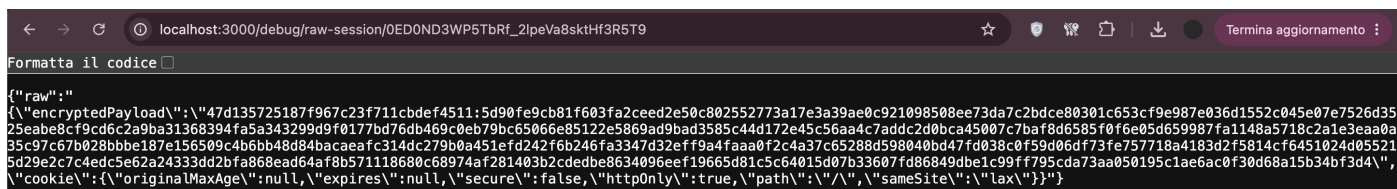
1. Avvio del server in modalità test tramite il comando: `NODE_ENV=test node server.js`;
2. Autenticazione sul portale e recupero dell'identificativo di sessione (`sid`) dal cookie `connect.sid` (estratto ripulendo la firma e il prefisso `s:`);
3. Interrogazione diretta dell'endpoint di debug tramite browser: `http://localhost:3000/debug/raw-session/<sid>`.

Come mostrato in Figura 14, nel sistema non protetto (o rimuovendo temporaneamente il wrapper crittografico) lo store restituisce i dati dell'utente loggato interamente in chiaro (inclusi ID utente, ruolo e impronte digitali). Nel sistema protetto, invece, lo store restituisce unicamente il payload cifrato in formato `IV:ciphertext`, confermando che a riposo nessuna informazione sensibile è leggibile in chiaro da un potenziale attaccante con accesso al dump della memoria o dello store delle sessioni.



```
localhost:3000/debug/raw-session/qOWN_G3Tnsc1Jrk7V9qRr43_OwbkZtAa
Formatta il codice
{"raw":{"cookie":{"originalMaxAge":null,"expires":null,"secure":false,"httpOnly":true,"path":"/","sameSite":"lax"},"userId":4,"role":"doctor","clientIp":"::1","clientUserAgent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36"}}
```

(a) Store delle sessioni nella versione vulnerabile. I dati della sessione risultano memorizzati in chiaro e sono direttamente leggibili da un eventuale attaccante che riesca ad accedere allo store.



```
localhost:3000/debug/raw-session/0ED0ND3WP5TbRf_2lpeVa8sktHf3R5T9
Formatta il codice
{"raw":{"encryptedPayload":"47d135725187f967c23f711cbdef4511:5d90fe9cb81f603fa2ceed2e50c802552773a17e3a39ae0c921098508ee73da7c2bdce80301c653cf9e987e036d1552c045e07e7526d3525eabe8cf9cd6c2a9ba31368394fa5a343299d9f0177bd76db469c0eb79bc65066e85122e5869ad9bad3585c44d172e45c56aa4c7addc2d0bca45007c7baf8d6585f0f6e05d659987fa1148a5718c2a1e3eaa0a35c97c67b028bbbe187e156509c4b6bb48d84bacaefc314dc279b0a451efd242f6b246fa3347d32eff9a4faaa0f2c4a37c65288d598040bd47fd038c0f59d06df73fe757718a4183d2f5814cf6451024d055215d29e2c7c4edc5e62a24333dd2bfa868ead64af8b571118680c68974af281403b2cdedbe8634096eef19665d81c5c64015d07b33607fd86849dbe1c99ff795cda73aa050195c1ae6ac0f30d68a15b34bf3d4","cookie":{"originalMaxAge":null,"expires":null,"secure":false,"httpOnly":true,"path":"/","sameSite":"lax"}}}}
```

(b) Store delle sessioni nella versione protetta. Il payload della sessione viene cifrato tramite AES-256-CBC prima della memorizzazione, rendendo non leggibili le informazioni sensibili presenti nella sessione.

Figura 14: Confronto tra la memorizzazione delle sessioni nella versione vulnerabile e nella versione protetta dell'applicazione. Nel primo caso i dati sono salvati in chiaro, mentre nel secondo risultano cifrati prima di essere scritti nello store.

8.3 Terza Sfida: Controllo di Concorrenza

La terza sfida consiste nel garantire che per ogni account esista una sola sessione valida alla volta (*Single Active Session*), prevenendo la condivisione impropria delle credenziali e riducendo l'impatto di possibili fenomeni di session hijacking.

Per risolvere questa problematica abbiamo modificato il ciclo di vita della sessione integrando le seguenti soluzioni:

- **Database Schema:** È stata aggiunta la colonna `active_session_id` TEXT alla tabella `users` nel file `db.js`.
- **Gestione del Login:** Ad ogni login andato a buon fine in `server.js`, il server aggiorna la tabella degli utenti salvando il nuovo identificativo di sessione (`req.sessionID`) all'interno del campo `active_session_id`.
- **Middleware di Controllo:** È stato introdotto il middleware globale `sessionConcurrencyCheck`. Per ogni richiesta effettuata da un utente autenticato, il middleware interroga il database per verificare che l'ID della sessione corrente coincida con quello memorizzato. Se un successivo login ha aggiornato l'identificativo registrato nel database, la sessione precedente risulta non più valida: il middleware la distrugge lato server (`req.session.destroy`), cancella il cookie sul client e reindirizza l'utente alla pagina di login mostrando un messaggio specifico:

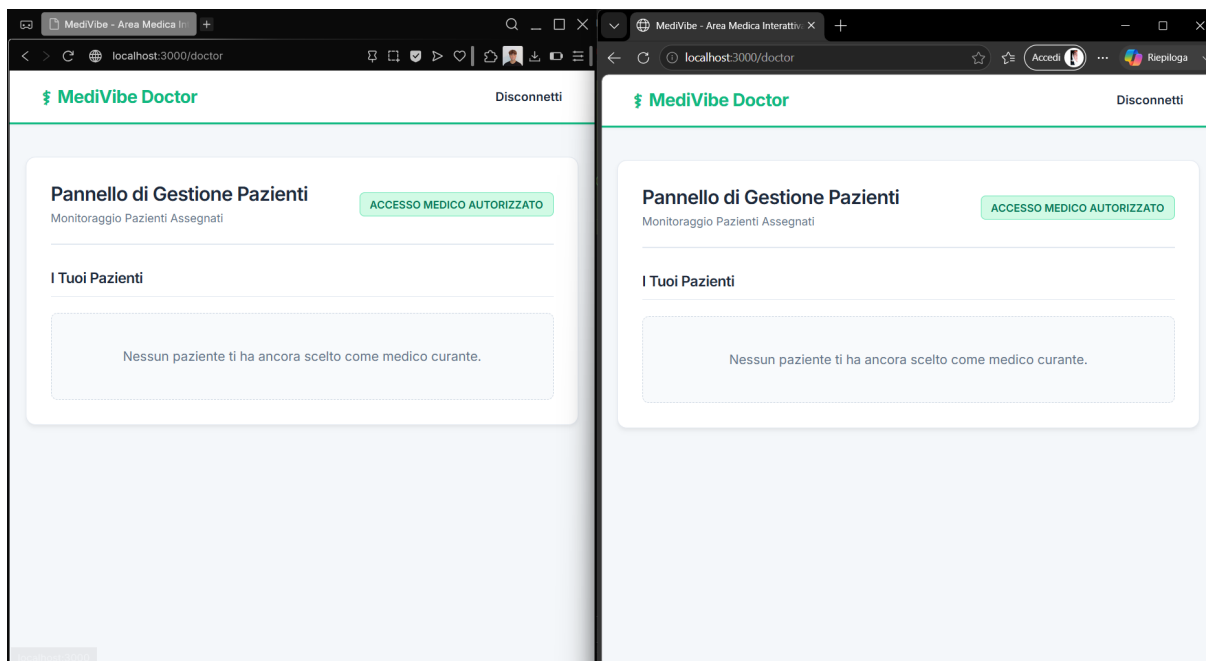
"Sessione terminata: è stato rilevato un accesso da un'altra postazione".

- **Disconnessione (Logout):** Durante la procedura di logout, il campo `active_session_id` viene reimpostato a `NULL` prima della distruzione della sessione, garantendo la corretta pulizia dello stato associato all'utente.

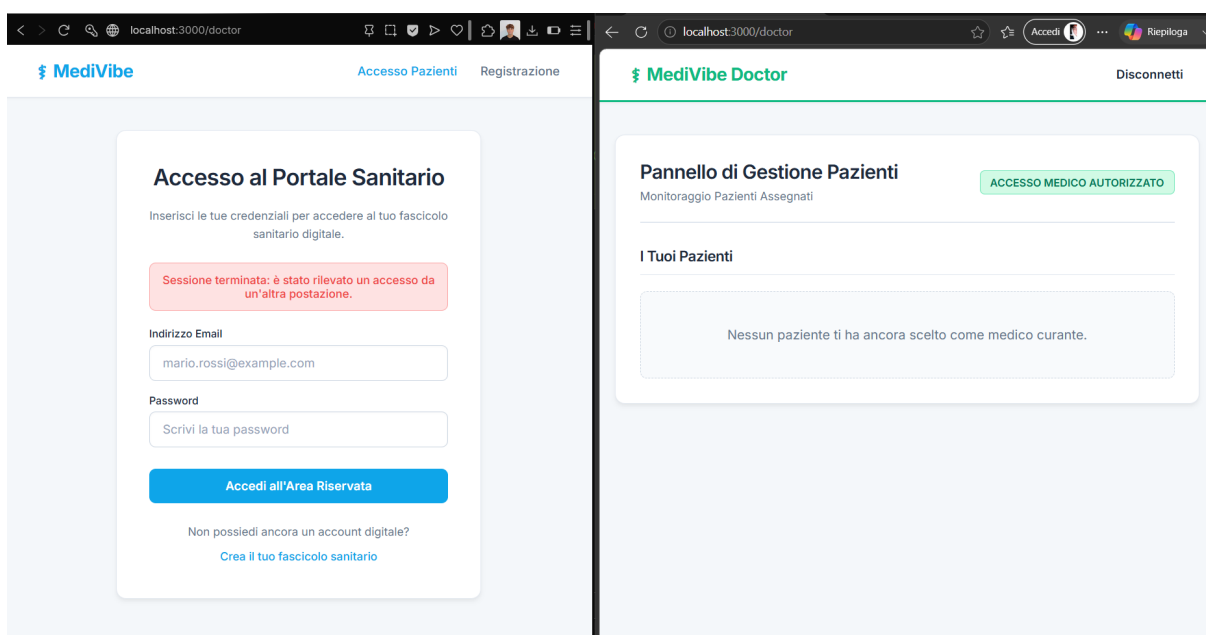
8.3.1 Verifica del Controllo di Concorrenza (Single Session Enforcement)

Per dimostrare il controllo di concorrenza, sono stati eseguiti due test manuali a confronto:

1. **Scenario Vulnerabile:** Sul branch vulnerabile (`initial-vulnerable-version`), è stato effettuato l'accesso allo stesso account medico contemporaneamente da due browser diversi. Come mostrato in Figura 15a, l'assenza di vincoli permetteva a entrambe le sessioni di rimanere attive e di operare in parallelo.
2. **Scenario Sicuro:** Sul branch sicuro, eseguendo l'accesso da un secondo browser con le stesse credenziali, il database memorizza il nuovo identificativo di sessione. Al successivo tentativo di interazione da parte del primo browser, il middleware rileva la discrepanza, invalida la sessione originaria sul server e reindirizza il client al login mostrando il messaggio di errore (come evidenziato in Figura 15b).



(a) Accesso simultaneo consentito con lo stesso account da due browser diversi nella versione vulnerabile.



(b) Controllo di concorrenza attivo: l'accesso da una nuova postazione provoca la disconnessione (logout) automatica dell'utente dal primo browser.

Figura 15: Confronto del controllo di concorrenza delle sessioni utente tra la versione vulnerabile e la versione protetta.